

Mobile-money app

Brainstorming conversation

18–19 September 2026 · Congo-Brazzaville

A chronological record of the discussion, from the initial France-to-Africa idea to a multi-country mobile-money product.

Latest decisions

- Prioritize mobile-money-to-mobile-money transfers.
- Launch domestic MTN ↔ Airtel transfers in Congo-Brazzaville first, subject to account and operating approvals.
- Design for multiple countries from the beginning.
- Use Congo-Brazzaville → Mali as the first proposed international expansion.
- Add bank-account-to-mobile-money later; Crédit du Congo and BGFIBank are candidates, and the founder has contacts at both.
- An existing pawaPay API account is available; actual enabled routes, costs, funding and partner permissions remain to be verified.

Reading note

The following pages preserve the conversation in order, including earlier proposals that were later changed or qualified. Figures and routes discussed as examples are not confirmed launch terms. Source links remain clickable. User wording is preserved; presentation is adapted for print. System messages, internal reasoning, tool logs and the PDF-export exchange are excluded.

01 · Original idea: cards, mobile money and France-to-Africa

You · 18 Sep 2026, 22:05 (UTC+1)

i have a new idea i want you to help me brainstorm it and help me ameliorate the idea from that,

i want o create an app to send money to some country, and send mobile money from different network, like in congo brazzaville and others country where pawapay is available, between MTN and Airtel, by using there carte bancaire, using the API Pawapay, also could send mobile money from mtn to airtel without using and banking card, personne from france can send money to a person in africa if the pawapay is available there

Assistant · 18 Sep 2026, 22:05 (UTC+1)

I'll sharpen this into a workable product concept, check the current pawaPay capabilities and country coverage, then separate the user experience from the regulatory and operational constraints that will shape the MVP.

Assistant · 18 Sep 2026, 22:07 (UTC+1)

Your idea is promising. The clearest version is:

A remittance app that lets people in France send money directly to a recipient's mobile-money wallet in Africa, even when the sender and recipient use different operators such as MTN and Airtel.

For example:

- Sender in France pays by bank card.
- They choose Congo-Brazzaville.
- They enter the recipient's phone number.
- Your app identifies whether the number belongs to MTN or Airtel.
- Your system sends the money to that wallet.
- The recipient receives local currency.

pawaPay appears suitable for the African payment side. It provides collections, payouts and remittances through mobile-money operators, and it lists Congo-Brazzaville among its payment-provider markets. Its API also supports selecting the recipient's mobile-money provider and checking transaction status.

[pawaPay product page](#) · [pawaPay payment providers](#) · [pawaPay remittance API](#)

The main improvement is to separate the product into two payment flows.

Use case	Sender pays with	Recipient receives
France → Africa	Bank card	MTN or Airtel wallet
Africa → Africa	MTN or Airtel wallet	MTN or Airtel wallet

For the France-to-Africa flow, pawaPay may handle the African payout, but you will probably need a separate card-payment provider for the French payment. Do not assume that pawaPay itself processes Visa or Mastercard payments. Confirm this directly with its sales team.

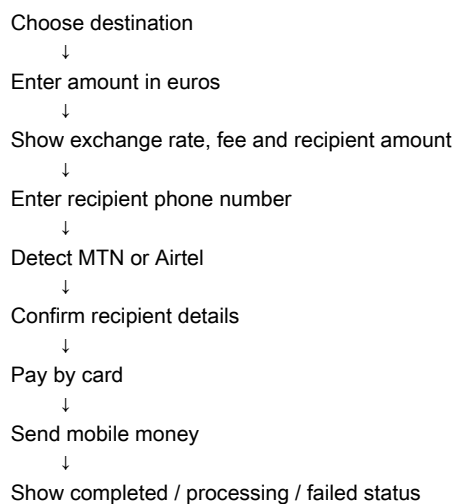
For the Africa-to-Africa flow, your application would collect money from the sender's mobile-money wallet and then make a payout to the recipient's wallet. This is technically possible in principle, but pawaPay must

confirm that the exact MTN-to-Airtel combination is enabled for your account and for Congo-Brazzaville.

I would start with this focused MVP:

1. France → Congo-Brazzaville.
2. Card payment in euros.
3. Delivery to MTN MoMo or Airtel Money.
4. Recipient phone-number validation.
5. Clear exchange rate, fees and delivery amount.
6. SMS, email or WhatsApp notifications.
7. Transaction tracking and automatic refund handling.
8. A support process for failed or delayed transactions.

The user experience could be:



A strong product promise could be:

“Send money from France directly to any supported mobile-money wallet in Africa.”

Your real competitive advantage should be trust and simplicity:

- show the final amount before payment;
- never hide the exchange rate;
- identify the recipient's network automatically;
- prevent sending to an incorrect operator;
- provide a real transaction reference;
- explain delays clearly;
- support French, English and possibly Lingala;
- allow repeat transfers to saved family members.

The most important business issue is regulation. If your company receives money from a customer in France and transfers it to another person, that is generally considered money remittance/payment activity.

In France, professional payment services normally require authorisation from the ACPR, unless your company operates through an appropriately authorised partner. [ACPR guidance](#) · [EU Payment Services Directive](#)

Therefore, you have three possible structures:

1. Become a regulated payment institution, which is expensive and slow.
2. Partner with a licensed remittance or payment institution in France and operate as its technology/distribution partner.
3. Use a regulated card-payment and remittance partner while your company focuses on the application, customer experience and marketing.

The third option is usually the best starting point. You should discuss with pawaPay whether they accept startups that are not themselves licensed remittance operators. Their current onboarding process includes business verification, KYB and a merchant agreement before production access. [pawaPay onboarding](#)

I would also avoid launching every country at once. Begin with one corridor:

France → Congo-Brazzaville → MTN/Airtel

Then measure:

- successful transfer rate;
- average delivery time;
- payment failure rate;
- customer acquisition cost;
- average transfer amount;
- fraud and chargeback rate;
- customer support cost.

Once this works reliably, add Belgium, Canada and more African destinations.

A refined business concept would be:

A simple family-remittance application for the African diaspora, allowing people in Europe to send money directly to MTN, Airtel and other mobile-money wallets, with transparent fees and fast delivery.

The next practical step is to produce a one-page product specification and contact pawaPay with three precise questions:

- Can pawaPay support France-funded remittances to MTN and Airtel wallets in Congo-Brazzaville?
- Can it support MTN collection followed by Airtel payout for domestic transfers?
- Which licensed partner must handle the French customer funds and card payments?

02 · Refocusing on Africa-to-Africa transfers

You · 18 Sep 2026, 22:09 (UTC+1)

but help me develop africa to africa, like sending money around different network, with an app

Assistant · 18 Sep 2026, 22:09 (UTC+1)

I'm narrowing the concept to Africa-to-Africa transfers: one person funds the transfer from a mobile-money wallet, and the recipient receives money on another network. I'll frame the first version around the transaction flow, the operator-routing problem, and the smallest MVP worth building.

Assistant · 18 Sep 2026, 22:11 (UTC+1)

Yes. The strongest version is an **Africa-to-Africa mobile-money bridge**:

A person sends money from their MTN wallet, and the recipient receives it on Airtel, or the reverse, through one app.

Example:

- Sender: MTN Congo wallet
- Recipient: Airtel Congo wallet
- Sender enters the recipient's phone number and amount
- Your app identifies the recipient's network
- The sender authorizes the debit with a mobile-money PIN or confirmation flow
- Your system pays the recipient on the other network

The user should not think about pawaPay. They should only see:

```
Send money
→ Choose country
→ Enter recipient phone number
→ Choose or detect MTN/Airtel
→ Enter amount
→ Review fee and final amount
→ Confirm
→ Receive transaction status
```

Technically, your app would work as an intermediary:

```
Sender mobile wallet
↓
pawaPay collection
↓
Your transaction engine
↓
pawaPay payout
↓
Recipient mobile wallet
```

pawaPay's API supports mobile-money collections, payouts and remittances. Its API examples include a recipient phone number, country, currency and mobile-money provider such as MTN_MOMO, and its configuration determines which operators, currencies and transaction limits are available for your account.

[pawaPay product documentation](#) · [pawaPay remittance API](#)

For the first version, I would focus on:

- Congo-Brazzaville only;
- MTN → Airtel;
- Airtel → MTN;
- amounts in XAF;
- domestic transfers;
- Android app plus a simple web administration panel.

This is a better starting point than launching many countries. You can test whether users actually need cross-network transfers and discover the real problems: failed transactions, wrong phone numbers, network downtime, limits and customer support.

Your main backend object should be a Transfer with:

```
transferId
senderPhone
senderProvider
recipientPhone
recipientProvider
country
amount
fee
recipientAmount
currency
collectionStatus
payoutStatus
pawaPayCollectionId
pawaPayPayoutId
createdAt
```

The transfer should follow a strict state machine:

```
CREATED
→ WAITING_FOR_SENDER_AUTHORIZATION
→ COLLECTION_PROCESSING
→ COLLECTION_COMPLETED
→ PAYOUT_PROCESSING
→ COMPLETED
```

If anything fails:

```
COLLECTION_FAILED
PAYOUT_FAILED
REFUND_REQUIRED
REFUNDED
```

Do not mark a transfer as successful just because pawaPay accepted your request. Wait for the final callback or status. pawaPay has states such as processing, enqueued, completed and failed, and an enqueued payment may need to wait for the operator network to recover. [pawaPay transaction statuses](#)

Your most valuable feature could be automatic recipient-network detection. The customer enters a phone number, and your app displays:

```
+242 06 123 45 67
Airtel Money Congo
```

The customer must confirm the number and network before authorizing the payment. This prevents one of the most damaging mistakes: sending money to the wrong wallet.

For your first business model, charge a small fixed fee plus a percentage:

Transfer amount: 10,000 XAF
Service fee: 100 XAF
Total sender pays: 10,100 XAF
Recipient receives: 10,000 XAF

You can later add:

- saved recipients;
- recurring family payments;
- transfer history;
- SMS or WhatsApp notifications;
- business payments;
- salary payments;
- school-fee payments;
- transfers between different African countries.

There are two important limits to validate before coding too much:

1. Whether pawaPay allows your account to collect from MTN and pay Airtel in the same country.
2. Whether the exact MTN/Airtel combinations are available in Congo-Brazzaville, including transaction limits and authorization requirements.

The product is therefore best described as:

An application that makes mobile-money networks interoperable for African users. Send from MTN, Airtel or another supported network and deliver directly to another mobile-money wallet.

For the development roadmap:

1. Create the user interface prototype.
2. Open a pawaPay sandbox account.
3. Confirm the MTN/Airtel collection and payout combinations.
4. Build the transfer backend.
5. Implement callbacks, refunds and reconciliation.
6. Test failed payments and operator downtime.
7. Launch a closed beta in Congo-Brazzaville.
8. Add other countries after the first corridor is reliable.

The key strategic decision is to begin with **domestic cross-network transfers** such as MTN Congo → Airtel Congo. Cross-border transfers such as Congo → Cameroon or Congo → Côte d'Ivoire require additional currency conversion, country limits and regulatory arrangements, so they should come after the first version works.

03 · Domestic market, bank funding and operating details

You · 18 Sep 2026, 22:21 (UTC+1)

i want much more details and informations before implementing, first we will focus on the domestic market, network to network and bank to network, i already have a pawapay API

Assistant · 18 Sep 2026, 22:21 (UTC+1)

That changes the scope in a useful way: we can design this as a domestic Congo-Brazzaville payment product with two funding methods—mobile-money-to-mobile-money and bank-to-mobile-money—before touching implementation. I'm checking the current pawaPay integration model and the local payment/regulatory landscape so the product plan reflects what your existing account can actually support.

Assistant · 18 Sep 2026, 22:23 (UTC+1)

Good. We should define the product as a **domestic payment-interoperability app for Congo-Brazzaville**, with two services:

1. **Mobile money** → **mobile money**

- MTN MoMo → Airtel Money
- Airtel Money → MTN MoMo
- Same country, usually XAF.

2. **Bank** → **mobile money**

- Bank account or bank payment → MTN MoMo or Airtel Money.
- Example: customer pays from a bank account, and the recipient receives money on MTN or Airtel.

The app's promise would be:

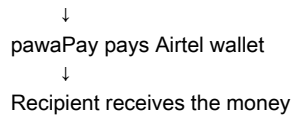
“Send money to any supported wallet in Congo, regardless of the network.”

Congo is a good first market because MTN and Airtel are the two major mobile operators, and mobile-money usage is already significant. The Congolese government currently reports approximately 3.7 million mobile-money subscriptions. [Ministry of Digital Economy](#) The Ministry of Finance also describes MTN MoMo and Airtel Money as payment institutions and mentions cooperation between networks as an objective. [Ministry of Finance](#)

1. How the mobile-money transfer should work

Example: MTN sender to Airtel recipient.

```
graph TD
    A[Sender enters recipient number] --> B[App detects Airtel network]
    B --> C[Sender enters 10,000 XAF]
    C --> D[App shows fee and recipient amount]
    D --> E[Sender confirms]
    E --> F[pawaPay collects from MTN wallet]
    F --> G[Your backend confirms collection]
```



The sender should authorize the collection through the method supported by pawaPay and the operator, such as a USSD confirmation, mobile prompt, PIN authorization or redirect flow. Your app should never ask for or store the sender's mobile-money PIN.

pawaPay should be treated as the payment infrastructure behind your application. Your application manages the customer experience, transfer records, pricing, risk controls and support. pawaPay manages the connection to the mobile-money providers.

A transfer should never be considered complete merely because pawaPay accepted the request. Your system should wait for the final status. pawaPay documents statuses such as ACCEPTED, SUBMITTED, PROCESSING, ENQUEUED, COMPLETED and FAILED. [pawaPay transaction statuses](#)

2. How bank-to-mobile-money should work

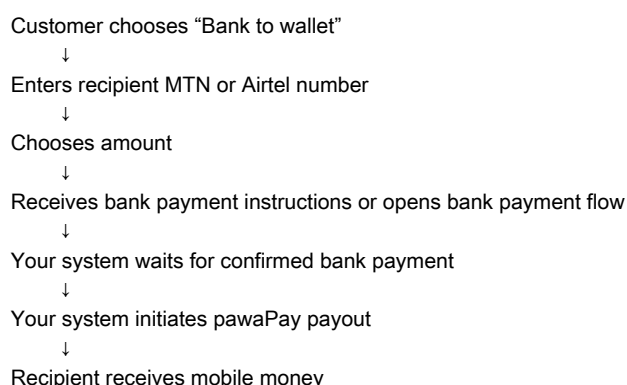
"Bank to network" should be called **bank-to-wallet** in the product documentation.

There are three possible bank models:

Model	How the customer pays	Difficulty
Bank card	Visa or Mastercard	Medium
Bank transfer	Customer sends money from a bank account	High
Bank-app integration	Your app connects to a bank API	Very high

For the first version, bank card is usually easier technically, but it introduces card fees, chargebacks and fraud. Bank transfer is more local and may be trusted by Congolese users, but it requires bank partnerships and reliable payment confirmation.

A bank-to-wallet transaction might work like this:



You must not release the mobile money simply because the customer says they made a bank transfer. Your system needs a confirmed bank notification, bank API response or verified reconciliation file.

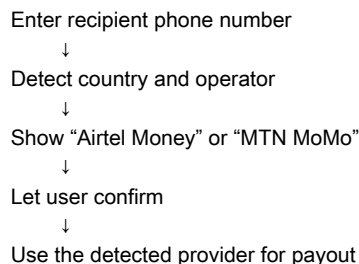
MTN Congo already advertises bank-to-wallet services through partners such as BGFI Mobile and MUCODEC, so there is a local precedent for this product category. [MTN Congo services](#)

For your MVP, I would make mobile-money-to-mobile-money the main service and add bank-to-wallet through one bank partner after the first flow is stable.

3. The most important product decision

You should not allow the user to choose an invalid operator combination.

Instead, your app should use this process:



The user may optionally select the recipient operator manually, but automatic detection should be the default.

Every operator combination should be stored as a configuration:

Country	Source provider	Destination provider	Currency	Status
Congo	MTN MoMo	Airtel Money	XAF	To validate
Congo	Airtel Money	MTN MoMo	XAF	To validate
Congo	MTN MoMo	MTN MoMo	XAF	To validate
Congo	Airtel Money	Airtel Money	XAF	To validate
Congo	Bank partner	MTN MoMo	XAF	To validate
Congo	Bank partner	Airtel Money	XAF	To validate

Do not assume that because pawaPay supports MTN and Airtel individually, every combination is automatically enabled for your account. Your account configuration, operator contracts, transaction limits and collection authorisation method determine what is available. pawaPay documentation says the active configuration controls supported providers, currencies, decimal rules and limits.

4. What your app actually needs to manage

Your backend should maintain one internal transaction record for every transfer.

A useful internal model is:

```
Transfer
- internalTransferId
- customerId
- senderPhone
- senderProvider
- recipientPhone
- recipientProvider
- country
- currency
- requestedAmount
- fee
- totalDebited
- amountToRecipient
- collectionId
- payoutId
- status
```

- failureReason
- createdAt
- completedAt

You should also keep a separate ledger. The ledger records every financial movement:

Customer paid: 10,100 XAF
 Service fee: 100 XAF
 Amount to recipient: 10,000 XAF
 pawaPay collection: 10,100 XAF
 pawaPay payout: 10,000 XAF

This prevents problems when the app database says “completed” but the actual pawaPay wallet balance does not match.

Your application should also reconcile regularly against pawaPay statements. pawaPay provides wallet balances and financial statements for reconciliation. [pawaPay wallet balances](#)

5. Recommended transfer statuses

Use internal statuses that describe the complete business process:

CREATED
 WAITING_FOR_COLLECTION
 COLLECTION_PROCESSING
 COLLECTION_COMPLETED
 PAYOUT_PROCESSING
 COMPLETED
 COLLECTION_FAILED
 PAYOUT_FAILED
 REFUND_PENDING
 REFUNDED
 MANUAL_REVIEW

The important rule is that collection and payout are two separate operations.

For example:

Collection completed
 Payout failed

In that case, you have received the sender’s money but the recipient has not received the funds. Your system must either retry the payout safely or refund the sender. This is one of the most important parts of the product.

Never retry blindly. Every request needs an idempotency key so a network timeout cannot create two payouts.

6. Customer experience

The main screen could contain only four actions:

- Send money
- Receive money
- Transaction history
- Help

The sending process should be:

1. Enter recipient phone number.

2. Detect the operator.
3. Enter the amount.
4. Display the fee.
5. Display the exact amount the recipient will receive.
6. Show the recipient's masked name or phone number if available.
7. Confirm.
8. Authorize the sender's wallet.
9. Track the payment.
10. Notify both people.

The confirmation screen should say:

You are sending 10,000 XAF

From: MTN MoMo
To: Airtel Money
Recipient: 06 XXX XX 45
Fee: 100 XAF
Total: 10,100 XAF

The recipient will receive: 10,000 XAF

This confirmation step is essential because a wrong phone number or wrong network can create a difficult recovery process.

You should support French first, with English later. Depending on the target audience, Lingala and Kituba could become valuable for customer support and notifications.

7. Limits and pricing

You need your own limits in addition to the operator's limits.

For example:

Limit	Example starting rule
Minimum transfer	500 XAF
Maximum first transfer	50,000 XAF
Daily customer limit	200,000 XAF
Monthly customer limit	1,000,000 XAF
New recipient limit	Lower limit for first transaction
High-risk transfer	Manual review

Airtel Money Congo publicly lists transaction limits, including a maximum of 500,000 FCFA per transaction and a maximum balance of 5,000,000 FCFA. Your app still needs to respect the lower limit between your risk policy and the provider's limit. [Airtel Money Congo limits](#)

A simple pricing model could be:

Transfer amount: 10,000 XAF
Service fee: 100 XAF
Sender pays: 10,100 XAF
Recipient receives: 10,000 XAF

Before setting prices, calculate:

pawaPay collection fee
+ pawaPay payout fee
+ bank fee, if applicable
+ SMS or notification cost
+ fraud losses
+ customer support cost
+ your margin

Do not hide the fee inside the exchange rate or recipient amount. Transparency will be important for customer trust.

8. Bank-to-wallet should probably be phase two

The best rollout would be:

Phase 1: Mobile-money-to-mobile-money

- MTN → Airtel
- Airtel → MTN
- MTN → MTN
- Airtel → Airtel
- XAF only
- Congo only

Phase 2: Bank-to-wallet

- One bank partner
- Bank account → MTN
- Bank account → Airtel
- Verified payment confirmation
- Daily reconciliation

Phase 3: Business use cases

- Merchant payments
- Salary payments
- School fees
- Rent payments
- Supplier payments
- Bulk payouts

This gives you a clear customer problem while keeping the first technical and operational scope manageable.

9. Compliance and operating model

Having a pawaPay API account does not automatically mean your company is authorised to operate every type of money-transfer service. You need to confirm:

- whether your pawaPay account is allowed to serve domestic consumer transfers;
- whether your company must be registered as a payment institution or agent;
- who holds customer funds;
- who performs customer identification;
- who handles suspicious transactions;
- who handles refunds;
- where transaction records are stored;
- whether you need agreements with MTN, Airtel or a local financial institution.

The Congolese Ministry of Finance states that MTN MoMo and Airtel Money are payment institutions governed within the financial-services framework. That means your product should be structured with a qualified local compliance adviser and formal partner agreements before public launch. [Congolese Ministry of Finance](#)

Your app should include:

- phone-number verification;
- customer identity verification where required;
- transaction limits;
- duplicate-account controls;
- device and IP monitoring;
- velocity checks;
- suspicious-pattern detection;
- audit logs;
- customer dispute handling;
- refund procedures.

You should avoid holding customer balances inside your own app during the first version. The app should initiate a transfer and track it. Creating an internal wallet or stored balance would create additional regulatory and accounting complexity.

10. The pawaPay questions you need answered

Because you already have access to pawaPay, the next discovery step is to inspect your account configuration and ask pawaPay these exact questions:

1. Is Congo-Brazzaville enabled for both MTN MoMo and Airtel Money?
2. Can our account collect from MTN and pay out to Airtel?
3. Can our account collect from Airtel and pay out to MTN?

4. Are domestic remittances supported, or must we use separate deposit and payout APIs?
5. What authorization method is required for each operator?
6. What are the minimum and maximum amounts?
7. What are the daily and monthly limits?
8. How are failed collections refunded?
9. How are completed collections handled when a payout fails?
10. Can we use signed callbacks?
11. How quickly are pawaPay wallet balances settled?
12. What KYB, KYC and local licensing documents are required for production?
13. Can one pawaPay account support both mobile-money transfers and bank-funded payouts?
14. What is the expected delivery time during operator downtime?
15. Which transaction statuses should trigger retry, refund or manual review?

Your first practical deliverable should be a **payment corridor matrix** for Congo. It should contain every source and destination combination, the required API operation, the expected authorization method, fees, limits, and failure-handling rule.

The product should begin as:

HDMarket Transfer: a Congo-Brazzaville app that lets customers move money between MTN MoMo, Airtel Money and, later, partner bank accounts, with one simple interface and transparent fees.

The most important technical principle is this: your app is not simply “calling pawaPay.” It is managing a financial transaction that has two legs, two providers, possible failures, reconciliation requirements and customer-support consequences. Designing those rules before writing the app will save you major problems later.

04 · Designing roles, transfer rules and recovery

You · 19 Sep 2026, 18:48 (UTC+1)

let start designing thoses roles and really know how the app work, and handle everything

Assistant · 19 Sep 2026, 18:59 (UTC+1)

Let's define the app's operating rules before designing screens or writing code. I'm taking "roles" to mean both **who does what** and **the rules that control each transfer**.

The starting scope is Congo-Brazzaville, XAF, MTN MoMo ↔ Airtel Money, plus bank-funded payments to those wallets. We'll design both funding methods now, while treating bank connectivity and your pawaPay account permissions as items to verify.

One correction to the earlier plan: the proposed fees and limits were examples, not validated launch settings. Likewise, detecting a network does not prove that a wallet exists or confirm its owner's name.

1. What the customer is actually using

I recommend a transfer app where customers pay for each transaction individually. They do not maintain an extra spendable balance inside your app.

There are two primary actions:

- **Send to someone:** pay from your mobile wallet or supported bank method and deliver to another person's mobile wallet.
- **Move my money:** move money between your own wallets, or from your bank to your wallet.

Both actions use the same underlying transfer process. "Move my money" can offer saved personal accounts, but ownership still needs appropriate verification.

The recipient should not need to install the app. They receive money in their existing mobile-money wallet. The sender uses the app to authorize, track and obtain support for the transfer.

2. Roles and responsibilities

Role	What they can do	Important boundary
Sender	Create transfers, authorize payment, view receipts, request help	Can access only their own activity
Recipient	Receive funds; optionally view a secure status page	Cannot see the sender's sensitive information
Customer support	Investigate transfers, explain status, open recovery cases	Cannot edit amounts, mark payments successful or redirect funds
Operations	Investigate delayed transfers and propose controlled recovery	Cannot create arbitrary payouts
Finance	Reconcile money movements, manage payout funding, review refunds	Cannot alter financial history
Risk/compliance	Review identity and suspicious activity, apply restrictions	Cannot silently change recipients or transaction amounts
Administrator	Manage staff access and configuration	Sensitive financial actions require separate approval

Role	What they can do	Important boundary
Payment partners	Authorize collections, execute payouts and report results	Their enabled services and contracts determine available routes

A small team can share responsibilities, but sensitive actions should retain separation. For example, the person requesting an exceptional payout should not approve it.

Every staff action should record who acted, when, why and what changed.

3. The customer journey

A first-time sender verifies their phone number, creates an app PIN or enables biometrics, and completes the identity checks required by the approved operating model. Phone verification alone is not identity verification.

Then:

1. Choose **Send to someone** or **Move my money**.
2. Enter the recipient's number or select a saved recipient.
3. Confirm the recipient's mobile-money network.
4. Enter how much the recipient should receive.
5. Select a funding method.
6. Review the full quote.
7. Authorize the payment through the mobile operator or bank.
8. Follow the transfer's progress.
9. Receive a receipt when delivery is confirmed.

Use **"Recipient receives"** as the default amount field. Someone sending school money usually wants the recipient to receive a specific amount.

An illustrative confirmation screen:

Item	Example
Recipient	Saved label: "Maman"
Wallet	Airtel Money • masked phone number
Recipient receives	10,000 XAF
App transfer fee	200 XAF
Total charged through this flow	10,200 XAF
Funding source	Your MTN wallet

The fee is hypothetical. A saved label is not a verified account name. Any provider name lookup should be labelled separately.

Also distinguish wallet delivery from cash withdrawal: receiving 10,000 XAF in a wallet does not establish what an operator will charge for later cash-out.

4. Rules before taking any money

The app should check:

- The source and destination route is approved and enabled.
- The phone number has a valid format.
- The selected operator matches available provider information.
- The transfer fits applicable amount and customer limits.
- The customer has completed the necessary verification.
- The destination service is available according to the latest information.
- Sufficient payout funding is available or reserved.
- The quote is still valid.
- The transaction is not an accidental duplicate.

Some checks cannot guarantee success. A recipient may use their wallet balance elsewhere while the transfer is processing, for example. The app must still handle provider rejection after collection.

Once the sender authorizes a quote, **freeze the recipient, amount, funding source and fee**. Changing them requires a new quote and authorization.

Never collect the operator's wallet PIN or bank password in your own app. Your app's PIN is a separate credential.

5. How mobile-money-to-mobile-money works

The proposed flow is two coordinated payment operations, subject to pawaPay approving this use case for your account:

1. Create one internal transfer reference.
2. Reserve the necessary destination funding.
3. Request collection from the sender's wallet.
4. The sender authorizes through the operator's supported process.
5. Your backend verifies that collection completed for the expected amount and currency.
6. Your backend requests the recipient payout.
7. Verify the payout's final result.
8. Complete the transfer and issue its receipt.

The app should show plain-language progress:

Waiting for your approval → Payment received → Delivering money → Delivered

Closing the app must not stop the transfer. The server continues managing it, and reopening the app shows the current state.

A crucial money-management detail: **collecting money on MTN does not establish that the same funds are immediately available for an Airtel payout**. Your account's wallet structure, available balances and settlement arrangements need verification.

For illustration, if destination funding is 100,000 XAF and accepted transfers already reserve 80,000 XAF, only 20,000 XAF remains available for new commitments, before any safety buffer or fees.

6. How bank-to-wallet works

We need to distinguish two different products:

Funding method	Customer experience	What confirms funding
Bank transfer/account payment	Customer authorizes through a supported bank flow or sends to a designated account with a reference	Verified bank confirmation matched to the transfer
Bank card	Customer pays through an approved card checkout	Processor confirmation meeting the agreed release policy

Your pawaPay access can support the mobile-money side where enabled; the bank/card funding integration remains a separate capability to establish.

For bank transfers:

- Give each payment a reliable matching reference.
- Match incoming money against the expected sender, amount and reference.
- Treat screenshots and uploaded receipts as supporting evidence only.
- Send unmatched, underpaid or overpaid transfers to a resolution process.
- If money arrives after the quote expires, revalidate the transfer before delivery.
- Do not silently send a different amount because the customer paid the wrong amount.

For cards, delivery needs a separate fraud and dispute policy. A successful card payment can later be disputed; the business must decide who bears that loss with its payment partner.

Bank-to-wallet belongs in the product design now, but the app should display only funding methods that actually have an approved, working integration.

7. Failure handling is part of the product

Situation	Customer sees	System action
Sender declines or does not authorize	"Payment not completed"	Confirm collection outcome before closing
Collection succeeds; payout is still processing	"Payment received. Delivery in progress."	Continue checking the existing payout
A request times out	"We're checking your transfer"	Resolve the existing request; do not create another payment
Payout definitively fails	"Delivery failed. We're resolving your payment."	Apply the retry-or-refund policy
Provider result is uncertain	"Transfer under verification"	Investigate; block conflicting payout/refund actions
Refund is processing	"Refund in progress"	Track it as its own payment
Refund fails	"Refund delayed. Support is handling it."	Keep the obligation open and escalate

Situation	Customer sees	System action
Money was delivered to the confirmed but wrong number	"Delivered" with a recovery-help option	Request recovery; do not promise reversal

The most important rule is: **an unknown outcome is not a failed payment.**

Never refund a collection while a payout might still complete. Never retry with a new payout reference just because a response is late. Those mistakes can cause both a refund and a recipient payment, or two recipient payments.

Cancellation also depends on the stage:

- Before payment submission: cancel the request.
- During collection or payout: attempt cancellation only where supported, then wait for confirmation.
- After delivery: offer a recovery request, not a guaranteed cancellation.

8. Money records and reconciliation

Keep three separate records:

- **Transfer:** what the customer requested.
- **Payment operations:** collection, payout and any refund, including individual attempts.
- **Financial ledger:** immutable records of money received, owed, paid, refunded and charged.

This prevents one "failed" status from hiding the fact that money was already collected.

Reconciliation should compare your records with partner transactions and balances. Every difference becomes a case:

- Partner reports success but your app still shows pending.
- A payout appears without a corresponding authorized transfer.
- A collected amount differs from the quote.
- A fee differs from the agreed schedule.
- A refund remains unresolved.

Staff must correct errors through recorded adjustments, never by deleting history or manually changing a transfer to "paid."

9. Pricing and protection rules

Set prices only after receiving actual collection, payout, banking and refund costs.

For each route, calculate:

Customer fee – collection cost – payout cost – other variable costs = contribution per transfer

Other variable costs include notifications, expected losses and support. That contribution still needs to cover salaries, infrastructure and other fixed expenses.

My proposed consumer policy is to return the collected principal and your app's fee when your service cannot deliver, subject to confirmed payment outcomes. Check the economics because partners may retain charges.

Use configurable limits rather than invented fixed values. Limits should account for customer verification, provider restrictions, transaction frequency and risk. A new recipient or recently changed account may require extra confirmation.

Account recovery deserves particular attention: someone who gains control of a phone number should not automatically gain unrestricted access to saved funding methods.

10. What the team needs to see

The administration panel should prioritize actionable queues:

- Collected but not delivered.
- Uncertain provider outcomes.
- Refunds awaiting completion.
- Unmatched bank payments.
- Low destination funding.
- Reconciliation differences.
- Customer complaints and risk reviews.

Each transfer page should show its timeline, quote, verified provider results, financial entries and support history. Sensitive identity documents and payment details should be restricted by role.

Support should be able to answer: **Where is the money, what are we waiting for, and what happens next?**

11. What makes the design ready to implement

Before building, we should settle these concrete items:

Decision	Required outcome
pawaPay permissions	Confirmation that your account can serve the proposed domestic consumer-transfer model
Operator capabilities	Enabled collection, payout, refund, authorization and verification features
Bank funding	Selected bank-transfer or card partner and confirmation method
Operating structure	Clear responsibility for customer funds, verification, complaints and financial oversight
Pricing	Route-specific costs and sustainable fees
Recovery policy	Approved retry, cancellation, refund and escalation rules
Liquidity	Funding targets, reservations and replenishment process

Then the prototype should demonstrate both successful transfers and the difficult cases: delayed authorization, uncertain payout, refund pending, wrong recipient and unmatched bank payment. Those scenarios determine whether the app feels trustworthy when something goes wrong.

The first distinction to settle is: **when you say “bank to network,” do you mean paying with a bank card, transferring directly from a bank account, or supporting both?**

05 · Clarifying bank-account funding

You · 19 Sep 2026, 19:00 (UTC+1)

paying from bank account to the user mobile money account

Assistant · 19 Sep 2026, 19:01 (UTC+1)

Understood: **the sender pays directly from a bank account, and the recipient receives money in their MTN MoMo or Airtel Money wallet.** We'll call this **bank-account-to-mobile-money**.

Our domestic product now has these routes:

Sender pays from	Recipient receives on
MTN MoMo	Airtel Money
Airtel Money	MTN MoMo
Supported bank account	MTN MoMo
Supported bank account	Airtel Money

The recipient can be the sender themselves or another person.

For the bank-funded route, the intended experience is:

1. The sender enters the recipient's number and confirms their network.
2. They enter the amount the recipient should receive.
3. The app shows the fee and total bank payment required.
4. They choose a supported bank-payment method and authorize through their bank.
5. Your system receives reliable confirmation of the incoming payment.
6. Your system initiates the mobile-money payout through pawaPay.
7. The app confirms delivery when the payout completes.

There are two possible ways to connect the bank side:

Approach	Customer experience	What we need
Integrated bank payment	The app opens an approved bank authorization flow, then the customer returns to track delivery	A bank or payment partner that supports this integration
Bank transfer with a reference	The app displays payment instructions; the customer makes the transfer in their banking app	An appropriately arranged receiving account and reliable transaction matching

The integrated approach is the preferred experience **if a suitable partner supports it**. The reference-based approach is a possible alternative, but confirmation speed and payment matching determine how convenient it will be.

Your existing pawaPay API gives us a starting point for the mobile-money side. We still need to establish the bank connection; we should not assume pawaPay can debit a customer's bank account.

The bank-funded transfer needs several specific rules:

- **Never pay out based on a screenshot.** Funding must be confirmed through the bank or authorized payment partner.
- **Match the payment to one transfer.** Check its reference, amount, currency and available payer information.
- **Handle missing or incorrect references.** Keep unmatched money in a resolution process rather than guessing the recipient.
- **Handle underpayments, overpayments and duplicate payments.** Do not automatically convert them into additional mobile-money payouts.
- **Handle late bank payments.** If the transfer request expired, recheck the recipient, fees and availability before proceeding.
- **Return failed-transfer funds through the original funding route where supported.** Refund timing and any exceptions must be agreed with the banking partner.
- **Manage payout funding separately.** Money arriving in a bank account does not necessarily become immediately available in your pawaPay payout balance.

For customers, the progress should be simple:

Waiting for bank payment → Payment received → Sending to mobile money → Delivered

We should only promise a delivery time once we know both the bank's confirmation speed and the mobile-money payout performance.

Which bank in Congo-Brazzaville would you like to support first, and do you already have a business account or a partnership with it?

06 · Selecting possible banks

You · 19 Sep 2026, 19:01 (UTC+1)

credit du congo or BGFI bank

Assistant · 19 Sep 2026, 19:01 (UTC+1)

I'll check what Crédit du Congo and BGFIBank currently offer for bank-to-mobile-money transfers, and what each would require to connect your app.

Assistant · 19 Sep 2026, 19:02 (UTC+1)

BGFIBank is my first candidate to investigate, with Crédit du Congo as the second. That is based on their advertised mobile-money connections—not confirmed access for your app.

I found something important: bank-to-wallet services already exist locally.

Bank	What official operator pages advertise	What remains unconfirmed
BGFIBank Congo	BGFI Mobile → MTN MoMo; Airtel also lists BGFIBank as a banking partner	Whether your app can initiate these payments through a partner API
Crédit du Congo	Airtel lists Crédit du Congo as a banking partner	Third-party integration, MTN delivery and permitted recipient types

MTN describes the BGFI service as transferring from a customer's bank account to their MoMo account through BGFI Mobile. Airtel lists both banks for its bank-to-wallet service. These listings establish existing consumer services; they do **not** establish an API available to your company. [MTN's BGFI service](#) · [Airtel's banking partners](#)

This changes how we should position your app. Airtel currently advertises bank-to-Airtel transfers at **0 FCFA** on its banking page, subject to confirmation of current terms. Charging for the same basic service could be difficult. Your value should be a unified experience, sending to other people where permitted, clear tracking and dependable support. [Airtel banking tariffs](#)

For the design, there are two possible bank arrangements.

Option A: the bank funds a transfer, and pawaPay delivers it.

The proposed flow is:

1. Customer selects BGFIBank or Crédit du Congo.
2. They enter the recipient's MTN or Airtel number and amount.
3. Your app displays the complete quote.
4. The bank authorizes the payment through an approved flow.
5. Your backend verifies payment into the agreed receiving arrangement.
6. Your backend requests the mobile-money payout through pawaPay.
7. The customer receives a delivery receipt.

This preserves your planned pawaPay architecture. It requires the bank to support payment initiation or reliable incoming-transfer confirmation, plus an agreed arrangement for receiving customer funds.

Option B: the bank executes the complete bank-to-wallet payment.

If a bank offers an approved partner interface that handles both debit and delivery, your app could initiate and track that operation. **You would not also initiate a pawaPay payout**, because that would pay the recipient twice. pawaPay could still serve your MTN ↔ Airtel routes.

We should choose between these arrangements after reviewing the bank's actual partner offering.

The bank discussion needs concrete answers:

Question for the bank	Why it matters
Can an external app initiate an account payment with customer consent?	Determines whether payment can start inside your app
Can customers pay another person's wallet, or only their own linked wallet?	Determines whether this supports your intended use case
Is there an API, test environment and transaction-status service?	Determines whether reliable integration is possible
Does the bank handle the mobile-money delivery, or only the bank payment?	Determines pawaPay's role
How are completed payments confirmed and matched?	Prevents paying out against unverified funding
Who receives and holds the funds before delivery?	Establishes the operating and accounting responsibilities
What are the fees, limits, availability and settlement timing?	Determines pricing and delivery promises
Who resolves failed payments and refunds?	Prevents customers being passed between partners

My recommendation is to **design one common bank-payment journey with configurable bank connections**, then launch with whichever bank provides the strongest approved integration. BGFI's existing MTN connection makes it a sensible first conversation, but commercial access and reliability should decide the winner.

Do you already have a business account or a contact at either bank?

07 · Bank contacts and partnership outreach

You · 19 Sep 2026, 19:05 (UTC+1)

i do have contact in either banks

Assistant · 19 Sep 2026, 19:05 (UTC+1)

Having contacts at both banks gives us a practical way to validate the bank-funded part of the app before investing in implementation.

I would approach both with the same proposal, then compare their answers. **The objective is to secure an approved integration that lets a customer authorize a bank payment and deliver money to their own or another person's mobile-money wallet.**

You can send your contacts this message:

Bonjour,

Je prépare une application de transfert d'argent domestique au Congo-Brazzaville. Elle permettra notamment à un client de payer depuis son compte bancaire pour envoyer de l'argent vers son propre portefeuille MTN MoMo/Airtel Money ou celui d'un autre bénéficiaire.

Nous disposons déjà d'un accès à l'API pawaPay pour la partie mobile money et souhaitons étudier un partenariat bancaire avant de commencer l'intégration.

Votre banque propose-t-elle une solution permettant à une application partenaire d'initier un paiement avec l'autorisation du client, puis d'en recevoir une confirmation fiable ? Nous souhaitons également savoir si votre banque peut directement assurer le transfert vers le portefeuille mobile money.

Pourriez-vous nous mettre en relation avec les équipes banque digitale, partenariats et conformité afin de discuter des possibilités techniques, des conditions commerciales et du cadre de fonctionnement ?

Merci.

For the meeting, bring a simple example: **a customer sends 25,000 XAF from their bank account to someone else's Airtel wallet.** Ask the bank to explain precisely how it could support that transaction.

Topic	Answer we need
Customer authorization	Where and how does the customer approve the bank debit?
Recipient eligibility	Can they send to another person, or only their own linked wallet?
Integration	Is there a partner API, documentation and a testing environment?
Payment confirmation	How can our system verify the payment's amount, reference and final outcome?
Delivery	Does the bank deliver to mobile money, or should pawaPay handle it?
Funds and settlement	Where does the money go, and when is it available for payout?
Failed transfers	Who returns the money, through what process, and within what timeframe?

Topic	Answer we need
Commercial terms	What setup fees, transaction fees, minimum volumes and reserves apply?
Responsibilities	Who handles customer verification, monitoring, complaints and required approvals?

Ask for documentation and a named technical contact after the discussion. A verbal “yes, we have mobile banking” does not establish that your app can use it.

Meanwhile, we can settle these bank-transfer rules in the product design:

- **One transfer has one delivery path.** Either the bank delivers the mobile money or pawaPay does.
- **The sender approves an exact payment.** Recipient, amount and fees are fixed before authorization.
- **Bank credentials stay with the bank.** Your app records authorization references, not banking passwords.
- **A confirmed debit and a confirmed delivery are separate events.** The app tracks both.
- **An uncertain result pauses recovery actions.** We check the existing payment before attempting another payout or issuing a refund.
- **Every transfer remains traceable.** Its bank payment, mobile-money delivery and any refund share an internal reference.
- **Available banks are enabled individually.** We only offer a bank once its integration and operating arrangements are validated.

The strongest partner will be the one that supports **payments to other people’s wallets, reliable automated confirmation, clear settlement and a workable refund process.** Those answers will determine the bank-payment screens and the responsibilities behind them.

08 · Mobile money first; multi-country vision

You · 19 Sep 2026, 19:08 (UTC+1)

for now we will focus on the the mobile money to mobile first and the bank to mobile after, also i want the app to be multi country app, that can be use in many country, and send money between the country also

Assistant · 19 Sep 2026, 19:09 (UTC+1)

Understood. We'll prioritize **mobile-money-to-mobile-money transfers**, design the app for multiple countries from the beginning, and add bank-to-mobile-money later.

The product becomes:

An app that lets people send money from their existing mobile-money wallet to another supported wallet, within their country or across supported African countries.

Multi-country availability and international transfers are separate capabilities. Offering domestic transfers in Congo and Cameroon does not automatically enable Congo → Cameroon. Each direction needs its own provider support, commercial approval, funding arrangements and applicable regulatory review. Our earlier assumption that pawaPay country coverage alone could enable transfers between those countries was too broad.

We should structure the product around the following decisions.

1. One app, three types of transfer

Transfer type	Example	Product treatment
Between networks domestically	MTN Congo → Airtel Congo	Initial launch priority
Between countries	A wallet in Congo → a supported wallet in another country	Enabled individually after validation
Between your own wallets	Your MTN wallet → your Airtel wallet	Same payment process, saved personal accounts

The recipient should receive money in their existing wallet without needing your app.

Bank funding remains a future payment method within this design.

2. Design internationally; activate routes gradually

We can build the first experience around Congo-Brazzaville while making country, currency, network, fees and verification requirements configurable.

The app should display only routes that are actually available. It should never show every pawaPay country as an available destination merely because it appears in a coverage list.

Each route needs its own configuration:

Route setting	What it controls
Origin country and network	Where the sender can pay
Destination country and network	Where the recipient can receive
Sending and receiving currencies	Whether currency conversion is needed

Route setting	What it controls
Collection and delivery method	Approved partner operations
Fees and exchange-rate source	The customer's quote
Transaction limits	What amounts are permitted
Customer-information requirements	What must be collected before authorization
Funding availability	Whether we can fulfill the payout
Delivery estimate	What timing we can reasonably communicate
Recovery policy	How failures and refunds are handled
Availability status	Whether the route accepts new transfers

Congo → another country and that country → Congo are two separate routes. Enabling one must not automatically enable the other.

3. The customer journey

The app can suggest the sender's country from their verified phone number, but customers must confirm the wallet they will pay from. Their current physical location should not silently determine the funding country.

The transfer experience is:

1. Choose your funding wallet.

For example, a saved MTN Congo wallet.

2. Choose the recipient's country and number.

Suggest the network where reliable information is available; request confirmation.

3. Enter the amount.

Let the sender choose either "I pay" or "Recipient receives."

4. Review the quote.

Show the total debit, recipient amount, currencies, fees, any exchange rate and estimated delivery time.

5. Complete required checks.

Request any additional information before collecting money.

6. Authorize through the mobile-money provider.

7. Track delivery and receive a receipt.

A saved recipient should include their country, phone number and wallet network. Before every repeat payment, recheck route availability rather than assuming previous details remain valid.

4. Cross-border transfers add a currency and funding step

For a domestic transfer, the sender and recipient amounts can both be in XAF. For an international transfer, the currencies may differ.

The customer should see:

Quote field	Meaning
Sender pays	Total amount debited in the sending currency
Transfer fee	Your disclosed charge
Exchange rate	Rate used when currencies differ
Recipient receives	Exact promised wallet credit in the destination currency
Quote expiry	How long those terms remain available

We need an agreed provider for currency conversion and settlement. We must not assume pawaPay automatically converts a collection in one country into spendable payout funds in another.

My recommended rule is to **honor the accepted recipient amount once funding succeeds within the agreed authorization window**. If money arrives after that window, follow an explicit late-payment policy: obtain agreement to a new quote or arrange a refund. Never silently reduce the recipient's amount.

Even when two countries use the same currency, the transfer still needs approval and funding arrangements for both countries.

5. The transfer engine stays consistent

Every route follows the same business sequence:

Check eligibility → Create quote → Reserve payout funding → Collect money → Confirm collection → Deliver money → Confirm delivery

Currency conversion and settlement fit into this process according to the partner arrangement.

The app must maintain separate collection, payout and refund records under one customer-facing transfer reference. This is especially important internationally: money collected successfully in one country does not mean the recipient was paid in the other.

The existing protection rules still apply:

- A timeout means **unknown**, not automatically failed.
- Check the original operation before creating another payment.
- Never issue a refund while the payout could still complete.
- Closing the app does not interrupt the server-side transfer.
- A route outage can stop new transfers while existing ones continue through resolution.
- Staff cannot change the recipient after authorization to "fix" a failed payout.

6. Operations must work country by country

The management panel should let authorized staff filter by country, network, currency and transfer direction.

It needs to show:

- Available and reserved payout funds.
- Collections received but not delivered.

- Delayed or uncertain transactions.
- Refunds awaiting completion.
- Differences between internal records and partner statements.
- Route-specific success rates, delivery times and costs.

A country outage should not stop the whole app. For example, we should be able to pause new payouts to one network while other approved routes remain available.

Customer verification, limits and support procedures should also be configurable by route. We should not apply one universal rule to every country without validating it.

7. A practical release sequence

Release	Customer capability
First	Congo-Brazzaville: MTN ↔ Airtel
Second	One approved international route from Congo to a selected destination
Third	The reverse international direction, if separately approved
Fourth	Additional domestic markets and international routes
Later	Bank-account funding for selected routes

This preserves your multi-country vision from the start while giving us a manageable first live service.

The next design decision is **which country should be the first destination outside Congo-Brazzaville**. That choice determines the currencies, networks, funding requirements and international customer journey we need to validate.

Which country do you want to connect with Congo first?

09 · Mali as the first international destination

You · 19 Sep 2026, 19:12 (UTC+1)

Mali

Assistant · 19 Sep 2026, 19:12 (UTC+1)

I'll check Mali's supported mobile-money networks and what we need to validate for a Congo-Brazzaville → Mali transfer.

Assistant · 19 Sep 2026, 19:14 (UTC+1)

Mali will be the first international destination in our design: Congo-Brazzaville → Mali. We'll keep Congo's domestic MTN ↔ Airtel transfers as the first launch priority and bank funding for later.

I checked pawaPay's public information and found an important inconsistency: its status page references **Orange and Moov services in Mali**, but the public provider table I reviewed does not list Mali. This suggests those services exist on the platform, but it does **not** confirm which operations your account can use. We need your account configuration and pawaPay's confirmation before promising either network. [pawaPay status](#) · [Provider documentation](#)

Our proposed route matrix is therefore:

Sender wallet	Recipient wallet	Current design status
MTN Congo	Airtel Congo	Initial domestic route; account approval to verify
Airtel Congo	MTN Congo	Initial domestic route; account approval to verify
MTN Congo	Orange Mali	Candidate international route
Airtel Congo	Orange Mali	Candidate international route
MTN Congo	Moov Mali	Candidate international route
Airtel Congo	Moov Mali	Candidate international route

Mali → Congo and domestic transfers within Mali remain separate future activations.

For a Congo → Mali transfer, the customer experience should be:

1. Select the funding wallet in Congo.
2. Choose Mali and enter the recipient's number.
3. Confirm the supported recipient network.
4. Enter the amount the recipient should receive.
5. Review the amount paid, fee, currencies and delivery estimate.
6. Authorize the collection through MTN or Airtel.
7. Track the transfer until the recipient's wallet is credited.

We must distinguish the two CFA currencies. Congo uses XAF; Mali uses XOF. The interface should display “FCFA — XAF” and “FCFA — XOF” where necessary, rather than treating them as one interchangeable balance. BEAC explicitly distinguishes XAF from the West African CFA currency, XOF. [BEAC explanation](#)

The business must obtain an agreed conversion and settlement arrangement. A successful XAF collection in Congo does not automatically provide XOF funding for delivery in Mali.

For this first international route, I would add these operating rules:

Rule	How it affects the app
Destination funding is reserved before collection	Avoid taking money for a payout we cannot fund
The accepted quote fixes the recipient amount	No silent changes after successful, timely funding
Sender and recipient information is collected before payment	Avoid discovering missing requirements after taking money
Each network can be paused separately	An Orange issue need not disable an approved Moov route
Uncertain payouts remain under investigation	Prevent duplicate payments or premature refunds
Refunds are tracked separately	“Refund requested” must not appear as “Refund received”

pawaPay’s remittance API includes sender and recipient details, original currency and amount, and transfer-purpose information. That means our international flow needs more than two phone numbers and an amount; the exact required information and operation must follow the approved arrangement.

[Remittance documentation](#)

The next validation with pawaPay should focus on five questions:

- Can our company operate **Congo-origin, mobile-money-funded consumer transfers to Mali** under the proposed arrangement?
- Are Orange Mali and Moov Mali enabled for incoming remittances on our account?
- Which collection and remittance operations should we use?
- Who handles XAF-to-XOF conversion, settlement and destination prefunding?
- What information, fees, limits and refund procedures apply?

We now have a concrete product scope: **one multi-country app, Congo domestic transfers first, Congo → Mali as the first international expansion, and bank-account funding later.** The next design artifact should use these routes to define the sender screens, transfer states and operations dashboard—including failed and delayed transfers.